

Senior Go fejlesztő – Tesztfeladat

Sep 30, 2026 · @Anita

Bevezető

Köszönjük az interjút! A következő lépés egy otthon elkészíthető feladat, amely a RackForest Cloud mindennapi problémáihoz hasonlít: aszinkron műveletek, párhuzamosság, hibatűrés, több-bérlős (multi-tenant) működés.

- **Időkeret:** kb. 5–6 óra munka. A beadásra a kiküldéstől számítva 7 nap áll rendelkezésre.
- **Prioritás:** egy működő, tesztelt mag többet ér, mint sok félkész funkció. Amit kihagysz, azt írd le a README-ben, és indokold.
- **AI-eszközök:** használhatod őket, de jelezd a README-ben, mire. A technikai beszélgetésen minden sort el kell tudnod magyarázni, és élőben módosítani.
- **Kérdések:** ha valami nem egyértelmű, kérdezz bátran e-mailben. Ha nem akarsz várni a válaszra, dönts, és írd le a feltételezésedet — ezt is értékeljük.

A feladat: Snapshot Service

Készíts Go-ban egy mikroszolgáltatást, amely virtuális gépek diszkjeiről (volume) kezel snapshotokat egy REST API-n keresztül.

A valóságban a snapshot a storage rétegen (nálunk Ceph RBD) készül: lassú, aszinkron, és néha hibázik. Ezt a réteget egy mock implementációval szimuláld az alábbi interface mögött:

```
type StorageBackend interface {  
    CreateSnapshot(ctx context.Context, volumeID, snapshotID string) error  
    DeleteSnapshot(ctx context.Context, volumeID, snapshotID string) error  
}
```

A mock késleltetése (pl. 2–10 mp) és hibaaránya (pl. 20%) legyen konfigurálható, a tesztekben pedig determinisztikusan vezérelhető.

Kötelező követelmények

1. **Aszinkron feldolgozás:** a létrehozó kérés azonnal visszatér, a snapshot a háttérben készül el.
2. **Párhuzamossági limit:** összesen legfeljebb N storage-művelet futhat egyszerre (konfigurálható, alapértelmezés: 3), és egy volume-on egyszerre legfeljebb egy.

3. **Idempotencia:** az `Idempotency-Key` headerrel ismételt POST ugyanazt a snapshotot adja vissza, nem hoz létre újat.
4. **Multi-tenancy és kvóta:** a bérlőt az `X-Tenant-ID` header azonosítja. Egy bérlő csak a saját snapshotjait látja és kezeli, és legfeljebb 10 nem törölt snapshotja lehet.
5. **Újrapróbálás:** storage-hiba esetén exponenciális backoff, legfeljebb 3 kísérlet, utána a snapshot hibás állapotba kerül (a hibaüzenettel együtt).
6. **Timeout:** egy storage-hívás legfeljebb 30 mp-ig futhat, `context` -tel megszakítva.
7. **Graceful shutdown:** SIGTERM-re a szolgáltatás nem fogad új kérést, a futó műveleteket megvárja egy konfigurálható határidőig, és utána is konzisztens állapotot hagy maga után.
8. **Tárolás:** in-memory tároló elég, de interface mögött, hogy később adatbázisra cserélhető legyen.

API és állapotgép

Minden végpont (a `/healthz` kivételével) megköveteli az `X-Tenant-ID` headert.

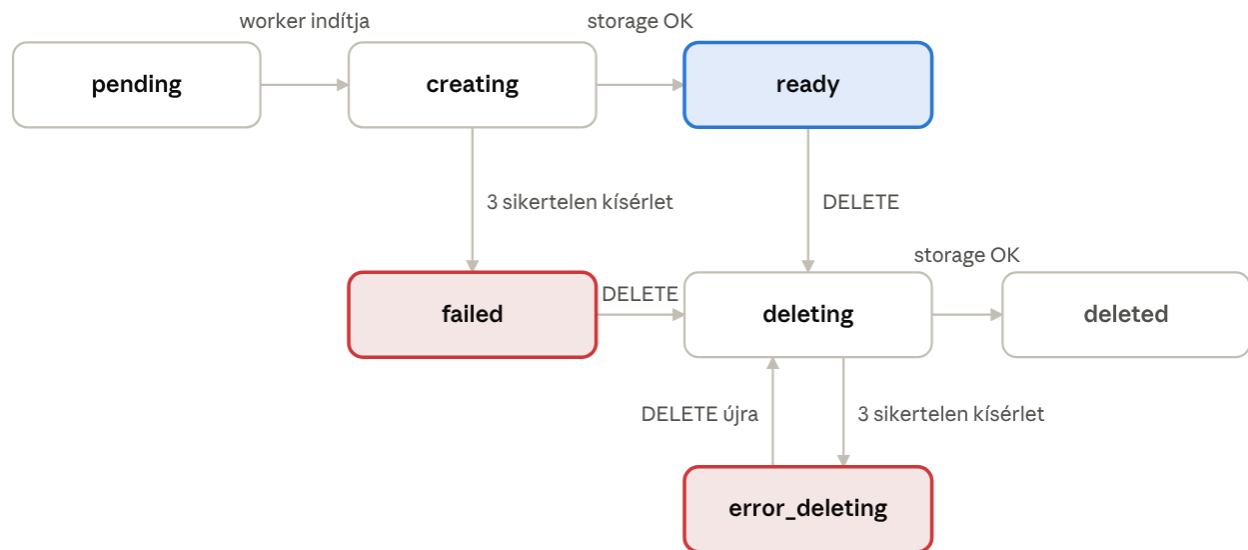
Metódus	Útvonal	Mit csinál	Sikeres válasz
POST	<code>/v1/volumes/{volumeId}/snapshots</code>	Új snapshot, body: <code>{"name": "..."} </code>	202 + snapshot (pending)
GET	<code>/v1/snapshots/{id}</code>	Egy snapshot lekérése	200
GET	<code>/v1/snapshots?volume_id=&status=</code>	A bérlő snapshotjai, szűrhetően	200
DELETE	<code>/v1/snapshots/{id}</code>	Aszinkron törlés, csak <code>ready</code> , <code>failed</code> vagy <code>error_deleting</code> állapotban	202
GET	<code>/healthz</code>	Élő-e a szolgáltatás	200

A snapshot objektum mezői: `id`, `tenant_id`, `volume_id`, `name`, `status`, `attempts`, `error`, `created_at`, `updated_at`.

A hibák formátuma egységes: `{"error": {"code": "...", "message": "..."} }`. A státuszkódokat (pl. kvótatúllépés, rossz állapotban kért törlés, idegen bérlő snapshotja) te választod meg — röviden indokold őket a README-ben.

Az állapotok és az átmenetek:

A snapshot életciklusa: hiba után törléssel lehet továbblépni



snapshot állapotgép · 7 állapot

A **deleted** snapshot már nem számít bele a kvótába. Más átmenet nem megengedett: például **creating** közben érkező DELETE kérést el kell utasítani.

Technikai elvárások és bónuszok

Kötelező

- Go 1.22 vagy újabb. A standard library előnyben (a `net/http` router is elég); külső csomag használható, ha a README-ben indokolod.
- Unit tesztek legalább az állapotátmenetekre, az idempotenciára és a párhuzamossági limitre.
- Legalább egy teszt, amely valódi párhuzamos terhelést szimulál, és hibátlanul fut `go test -race` alatt.
- Strukturált logolás (`log/slog`), kéresenkénti request ID-val.
- Konfiguráció környezeti változókból.
- Multi-stage Dockerfile, és egy paranccsal indítható futtatás és tesztelés (`make` vagy `docker compose`).
- README: indítás, a fő design-döntések és trade-offok, és hogy mit csinálnál másképp éles környezetben.

Bónusz (nem kötelező, ne ezzel kezd)

- Prometheus metrikák a `/metrics` végponton (pl. sorhossz, művelet-időtartam, hibaszám).
- PostgreSQL tároló migrációval.
- Újraindítás utáni helyreállítás: a `creating` vagy `deleting` állapotban ragadt snapshotok kezelése.
- OpenAPI specifikáció.

Írásos kérdések

Válaszolj röviden a README végén, összesen kb. egy oldalban. Nincs egyetlen jó válasz: a gondolatmenetedre vagyunk kíváncsiak.

1. **Skálázás:** a szolgáltatás 3 példányban fut egy load balancer mögött, közös adatbázissal. Mi romlik el a jelenlegi megoldásodban (párhuzamossági limit, volume-zárolás, idempotencia), és hogyan oldanád meg?
2. **Összeomlás:** a folyamat leáll, miközben egy snapshot `creating` állapotban van. Mi a helyzet újraindítás után? Hogyan kerülöd el, hogy árva snapshot maradjon a storage-on, vagy beragadt rekord az adatbázisban?
3. **Konzisztencia:** futó VM diszkjéről készül a snapshot. Mitől lesz crash-consistent, és mitől application-consistent? Mit tennél ennek érdekében egy KVM/QEMU alapú platformon?
4. **Visszaélés:** egy bérlő egy scriptből egyszerre 500 snapshotot kér. Milyen védelmet építenél be API- és platformszinten?

Beadás és a következő lépés

- Egy git repository linkjét várjuk (GitHub, GitLab vagy hasonló; privát repónál kérj tőlünk hozzáférési címet).
- A commit history maradjon meg, ne squash-old össze: a munka menete is érdekel.
- A README tartalmazza az írásos válaszokat és az AI-eszközök használatát.

A beadás után egy kb. 60 perces technikai beszélgetés következik:

1. Végigmész a kódon, és elmondod a fő döntéseidet.
2. Élőben módosítunk rajta egy-két dolgot (pl. egy új követelmény miatt).
3. Megbeszéljük az írásos válaszaidat.

Sok sikert, és jó munkát!